

23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : D8

Team Members :

Roll No	Student Name
AM.SC.U4CSE25352	SREYAK S
AM.SC.U4CSE25354	THARAK GOPAL
AM.SC.U4CSE25320	HARI SANKAR V
AM.SC.U4CSE25314	DHANANJAYAN SREERAJ

Project Title : **CinéArchive: A Database-Driven Film Discovery, Rating, and Curation Platform**

3.1. Detailed description of the project

1. Introduction & Overview

CinéArchive is a comprehensive, database-driven web platform designed for film discovery, logging, rating, and curation. Functioning as a digital community journal for cinema lovers, the platform allows

users to explore a vast catalog of films, record their viewing history, write detailed reviews, and curate customized movie lists. The system is managed through a robust relational database that tracks

interconnected data about media, user interactions, social connections, and community trends, alongside an administrative console for managing operations.

2. The Users

The platform caters to two primary types of users:

- Standard Users (Cinephiles, Reviewers, Casual Viewers): Registered members who browse the database, log films they have watched, maintain watchlists, write reviews, and follow other users to see their activity.*

- *Administrators: Privileged users who have access to the Database Administration Console. They are responsible for managing the platform's core catalog (adding new movies, actors, and directors),*

monitoring system triggers, managing database views, and executing stored procedures for analytics and recommendations.

3. Media & Metadata Catalog

At the heart of CinéArchive is the media catalog. The primary entity is the Movie, which stores essential details such as a unique identifier, title, synopsis, release year, runtime, average community

rating, and a calculated popularity score.

Because films are collaborative efforts, the database tracks the cast and crew. It maintains records of Actors and Directors, storing their names and unique IDs. A movie can have multiple actors (forming a

cast) and multiple directors, and conversely, actors and directors participate in many movies. When an actor is linked to a movie, the specific character name they played is also recorded.

To categorize the catalog, movies are linked to one or more Genres (e.g., Sci-Fi, Drama). Furthermore, the system tracks critically acclaimed cinema by recording Awards. Each award record notes the award

name, category, and the year it was won, and is linked directly to the movie and, if applicable, the specific director who received it.

4. User Interaction & Journaling

When a user interacts with a film, they create a Journal Log. A log represents a specific viewing event. Each log records the user, the movie, the date watched, whether the user "liked" the film (a simple

boolean), a star rating (e.g., 1 to 5), and an optional textual review.

Users can also maintain a Watchlist, which is a separate tracking list indicating films they intend to watch in the future. The watchlist records the user, the movie, and the date it was added. As users

continuously log films, the system aggregates their activity into a User Stats profile, which tracks their total films logged, total reviews written, and their average rating given across all films.

5. Curation & Custom Lists

A major feature of CinéArchive is content curation. Users can create custom, themed Lists (e.g., "Best Sci-Fi of the 2010s"). A list has a title, description, creation date, and a privacy flag (public or

private). A list can contain many movies, and a movie can belong to many lists. When movies are added to a list, the specific order in which they were added or ranked is recorded.

6. Social Connections & Notifications

CinéArchive includes a social networking layer. Users can follow each other to stay updated on their cinematic journeys. This creates a follower-followee relationship that tracks who is following whom and

the date the connection was made. To keep users engaged, the system generates Notifications to alert users of social interactions (e.g., "User X followed you" or "User Y liked your list"), storing the

notification message, a timestamp, and a read-status flag.

7. Specific Business Rules & Database Constraints

To maintain data integrity and enforce the platform's logic, the following business rules apply:

1. Unique Identification: Every user must have a unique username and email address. Every movie must have a unique Movie ID.

2. List Creation Constraint: A custom List cannot be successfully saved unless it contains a minimum of 2 films.

3. Dynamic Rating Updates (Triggers): A movie's AverageRating is not manually entered; it must be dynamically calculated and updated via a database trigger every time a user inserts, updates, or deletes a

rating in their Journal Log.

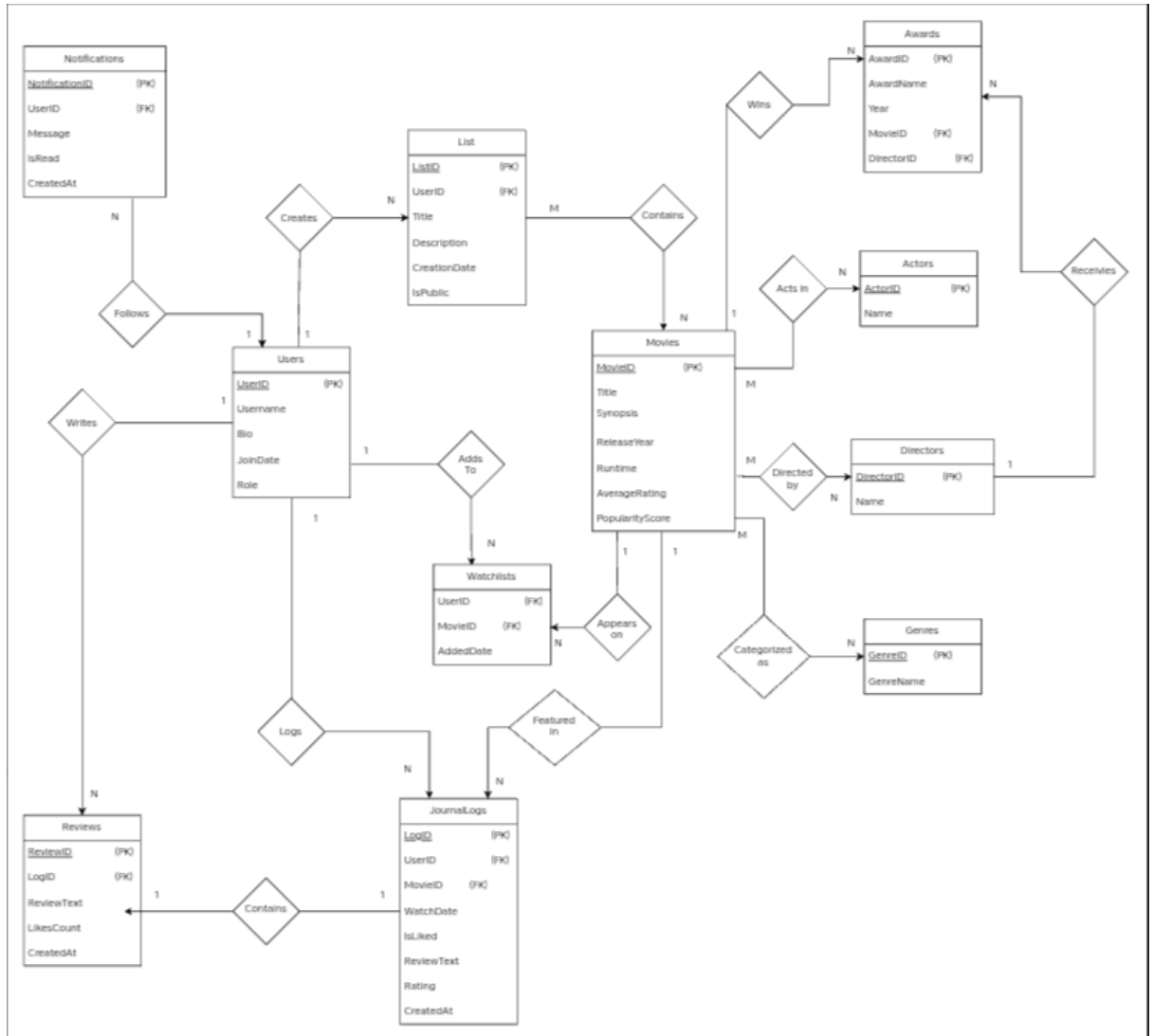
4. Social Constraints: A user cannot follow themselves. The combination of Follower and Followee must be strictly unique.

5. Watchlist Resolution: If a user logs a film in their Journal Log that currently exists on their Watchlist, the system should automatically remove that movie from the Watchlist.

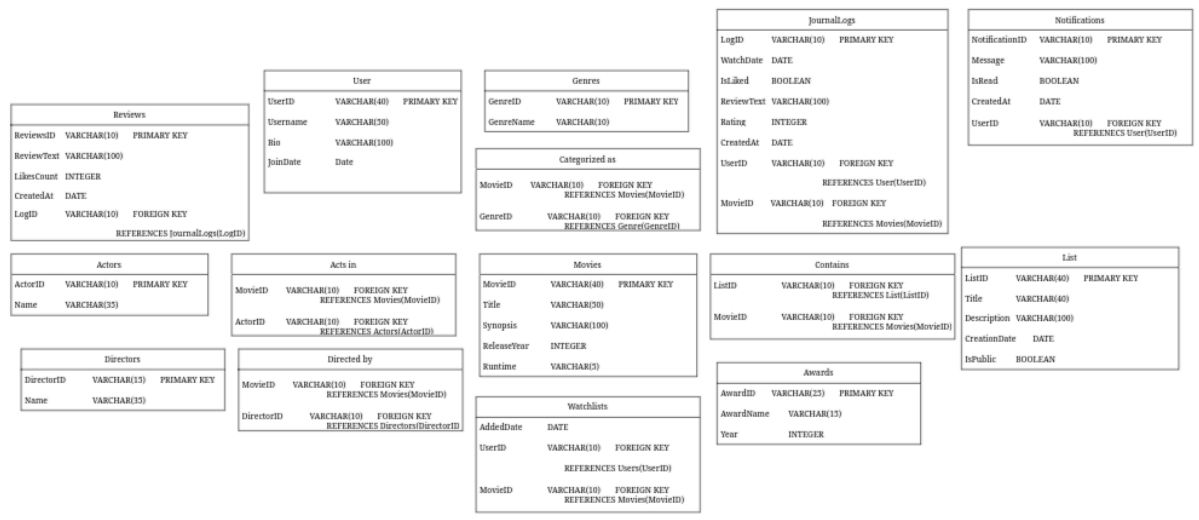
6. Analytics & Recommendations (Procedures): The system relies on stored procedures (e.g., `sp_generate_recommendations`) to analyze a user's logged genres and suggest unwatched movies, and predefined views

(e.g., *vw_movie_awards*) to seamlessly join *Movies*, *Directors*, and *Awards* for administrative display.

3.2. ER Diagram



3.3 Relational Schema



3.4 Normalization and Functional dependency analysis

3.4.1 Initial Unnormalized Form (UNF)

Initially, the CinéArchive database can be represented using a single conceptual relation containing information about users, movies, actors, directors, genres, journal logs, lists, watchlists, reviews, awards, and notifications.

CineArchive(UserID, Username, Bio, JoinDate, MovieID, Title, Synopsis, ReleaseYear, Runtime, ActorID, ActorName, CharacterName, DirectorID, DirectorName, GenreID, GenreName, LogID, WatchDate, IsLiked, Rating, ReviewText, ReviewID, ListID, ListTitle, Description, CreationDate, IsPublic, AwardID, AwardName, AwardYear, NotificationID, Message, IsRead, CreatedAt, ...)

This representation contains repeating and multivalued attributes. For example, a movie can have multiple actors, directors, and genres, while a user can have multiple journal logs, watchlist entries, and notifications. A list can also contain multiple movies.

Therefore, the above relation is not in First Normal Form (1NF).

3.4.2 Conversion to First Normal Form (1NF)

A relation is in 1NF when all attributes contain atomic values and repeating or multivalued attributes are eliminated.

For example, instead of storing multiple actors in a single attribute, each actor-movie association is stored as a separate tuple:

ActsIn(MovieID, ActorID, CharacterName)

Similarly, movie-genre relationships are represented separately:

CategorizedAs(MovieID, GenreID)

The resulting 1NF relations are:

User(UserID, Username, Bio, JoinDate)

Movies(MovieID, Title, Synopsis, ReleaseYear, Runtime)

Actors(ActorID, Name)

Directors(DirectorID, Name)

Genres(GenreID, GenreName)

Awards(AwardID, AwardName, Year)

List(ListID, Title, Description, CreationDate, IsPublic)

JournalLogs(LogID, WatchDate, IsLiked, Rating, ReviewText, CreatedAt, UserID, MovieID)

Reviews(ReviewID, ReviewText, LikeCount, CreatedAt, LogID)

Notifications(NotificationID, Message, IsRead, CreatedAt, UserID)

ActsIn(MovieID, ActorID, CharacterName)

DirectedBy(MovieID, DirectorID)

CategorizedAs(MovieID, GenreID)

Contains(ListID, MovieID)

Watchlists(UserID, MovieID, AddedDate)

*All attributes are now atomic and all repeating groups have been removed.
Therefore, the relations are in 1NF.*

3.4.3 Functional Dependencies

The major functional dependencies identified in the relations are as follows:

User:

$UserID \rightarrow Username, Bio, JoinDate$

$Username \rightarrow UserID, Bio, JoinDate$

Movies:

$MovieID \rightarrow Title, Synopsis, ReleaseYear, Runtime$

Actors:

$ActorID \rightarrow Name$

Directors:

$DirectorID \rightarrow Name$

Genres:

$GenreID \rightarrow GenreName$

Awards:

$AwardID \rightarrow AwardName, Year$

List:

ListID → *Title, Description, CreationDate, IsPublic*

JournalLogs:

LogID → *WatchDate, IsLiked, Rating, ReviewText, CreatedAt, UserID, MovieID*

Reviews:

ReviewID → *ReviewText, LikeCount, CreatedAt, LogID*

Notifications:

NotificationID → *Message, IsRead, CreatedAt, UserID*

ActsIn:

(MovieID, ActorID) → *CharacterName*

Watchlists:

(UserID, MovieID) → *AddedDate*

For DirectedBy, CategorizedAs, and Contains, the composite keys uniquely identify each relationship and there are no additional non-key attributes in the current schema.

3.4.4 Candidate Keys and Attribute Closure

Candidate keys are identified using attribute closure.

For Movies:

$MovieID \rightarrow Title, Synopsis, ReleaseYear, Runtime$

$MovieID^+ = \{MovieID, Title, Synopsis, ReleaseYear, Runtime\}$

Since the closure contains all attributes of the Movies relation, MovieID is a candidate key.

For ActsIn:

$(MovieID, ActorID) \rightarrow CharacterName$

$(MovieID, ActorID)^+ = \{MovieID, ActorID, CharacterName\}$

Therefore, (MovieID, ActorID) is a candidate key.

Neither MovieID nor ActorID alone can identify a tuple because a movie can have multiple actors and an actor can participate in multiple movies.

For Watchlists:

$(UserID, MovieID) \rightarrow AddedDate$

$(UserID, MovieID)^+ = \{UserID, MovieID, AddedDate\}$

Therefore, (UserID, MovieID) is a candidate key.

The candidate keys of the main relations are:

User: UserID, Username

Movies: MovieID

Actors: ActorID

Directors: DirectorID

Genres: GenreID

Awards: AwardID

List: ListID

JournalLogs: LogID

Reviews: ReviewID

Notifications: NotificationID

ActsIn: (MovieID, ActorID)

DirectedBy: (MovieID, DirectorID)

CategorizedAs: (MovieID, GenreID)

Contains: (ListID, MovieID)

Watchlists: (UserID, MovieID)

3.4.5 Conversion from 1NF to 2NF

A relation is in 2NF if it is already in 1NF and no non-prime attribute is partially dependent on a proper subset of a composite candidate key.

The relations with composite candidate keys are examined first.

ActsIn:

ActsIn(MovieID, ActorID, CharacterName)

Functional dependency:

$(MovieID, ActorID) \rightarrow CharacterName$

CharacterName depends on the complete combination of MovieID and ActorID. It does not depend on MovieID alone or ActorID alone.

Therefore, there is no partial dependency and ActsIn is in 2NF.

Watchlists:

$Watchlists(UserID, MovieID, AddedDate)$

Functional dependency:

$(UserID, MovieID) \rightarrow AddedDate$

AddedDate depends on the complete combination of UserID and MovieID. Therefore, there is no partial dependency and Watchlists is in 2NF.

The DirectedBy, CategorizedAs, and Contains relations contain only their composite keys and therefore cannot have partial dependencies.

All other relations have single-attribute candidate keys. Since partial dependency can occur only with composite candidate keys, these relations automatically satisfy 2NF.

Therefore, all relations are in 2NF.

3.4.6 Conversion from 2NF to 3NF

A relation is in 3NF if it is in 2NF and contains no transitive dependency of a non-key attribute on a candidate key.

For Movies:

Movies(MovieID, Title, Synopsis, ReleaseYear, Runtime)

MovieID $\rightarrow$ Title, Synopsis, ReleaseYear, Runtime

All non-key attributes depend directly on the candidate key MovieID. There is no transitive dependency.

Therefore, Movies is in 3NF.

For Actors:

ActorID $\rightarrow$ Name

The non-key attribute Name depends directly on ActorID. Therefore, Actors is in 3NF.

For Directors:

DirectorID $\rightarrow$ Name

The non-key attribute Name depends directly on DirectorID. Therefore, Directors is in 3NF.

For Genres:

$GenreID \rightarrow GenreName$

The non-key attribute GenreName depends directly on GenreID. Therefore, Genres is in 3NF.

For Awards:

$AwardID \rightarrow AwardName, Year$

Both non-key attributes depend directly on AwardID. Therefore, Awards is in 3NF.

For List:

$ListID \rightarrow Title, Description, CreationDate, IsPublic$

All non-key attributes depend directly on ListID. Therefore, List is in 3NF.

For JournalLogs:

$LogID \rightarrow WatchDate, IsLiked, Rating, ReviewText, CreatedAt, UserID, MovieID$

All non-key attributes depend directly on LogID. Therefore, JournalLogs is in 3NF.

For Reviews:

ReviewID → ReviewText, LikeCount, CreatedAt, LogID

All non-key attributes depend directly on ReviewID. Therefore, Reviews is in 3NF.

For Notifications:

NotificationID → Message, IsRead, CreatedAt, UserID

All non-key attributes depend directly on NotificationID. Therefore, Notifications is in 3NF.

The ActsIn, DirectedBy, CategorizedAs, Contains, and Watchlists relations also contain no transitive dependencies.

Therefore, all relations satisfy 3NF.

3.4.7 Final 3NF Schema

The final normalized schema is:

User(UserID PK, Username UNIQUE, Bio, JoinDate)

Movies(MovieID PK, Title, Synopsis, ReleaseYear, Runtime)

Actors(ActorID PK, Name)

Directors(DirectorID PK, Name)

Genres(GenreID PK, GenreName)

Awards(AwardID PK, AwardName, Year)

List(ListID PK, Title, Description, CreationDate, IsPublic)

JournalLogs(LogID PK, WatchDate, IsLiked, Rating, ReviewText, CreatedAt, UserID FK, MovieID FK)

Reviews(ReviewID PK, ReviewText, LikeCount, CreatedAt, LogID FK)

Notifications(NotificationID PK, Message, IsRead, CreatedAt, UserID FK)

ActsIn(MovieID PK/FK, ActorID PK/FK, CharacterName)

DirectedBy(MovieID PK/FK, DirectorID PK/FK)

CategorizedAs(MovieID PK/FK, GenreID PK/FK)

Contains(ListID PK/FK, MovieID PK/FK)

Watchlists(UserID PK/FK, MovieID PK/FK, AddedDate)

3.4.8 Summary of Normalization

The normalization process can be summarized as:

UNF

↓

Removal of repeating and multivalued attributes

↓

1NF

↓

Removal of partial dependencies

↓

2NF

↓

Removal of transitive dependencies

↓

3NF

The initial unnormalized relation contained repeating and multivalued attributes. These were separated into individual relations to obtain 1NF. The resulting relations were then checked for partial dependencies, and none were found. Therefore, the relations satisfy 2NF. Finally, the relations were checked for transitive dependencies, and no transitive dependencies were identified. Hence, the final relational schema satisfies 3NF.

The normalized design reduces data redundancy, prevents update anomalies, and maintains data integrity while preserving the relationships between users, movies, actors, directors, genres, awards, journal logs, reviews, lists, watchlists, and notifications.